

2. Using SVM, a company wants to classify emails as spam or not spam based on features such as word frequency and message length.

```
In [1]: # Importing Required Libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import classification_report
from sklearn import metrics
from sklearn.preprocessing import StandardScaler
import seaborn as sn
import matplotlib.pyplot as plt
```

```
In [2]: # Generating a Synthetic Email Spam Dataset
np.random.seed(42)
n_samples = 500

# Features: word_frequency, message_length, special_char_count, link_count
# Spam emails tend to have higher word frequency, longer messages, more special
spam_word_freq = np.random.normal(70, 15, n_samples // 2)
spam_msg_len = np.random.normal(300, 80, n_samples // 2)
spam_special_chars = np.random.normal(15, 5, n_samples // 2)
spam_link_count = np.random.normal(5, 2, n_samples // 2)

ham_word_freq = np.random.normal(35, 12, n_samples // 2)
ham_msg_len = np.random.normal(150, 60, n_samples // 2)
ham_special_chars = np.random.normal(5, 3, n_samples // 2)
ham_link_count = np.random.normal(1, 1, n_samples // 2)

word_frequency = np.concatenate([spam_word_freq, ham_word_freq])
message_length = np.concatenate([spam_msg_len, ham_msg_len])
special_char_count = np.concatenate([spam_special_chars, ham_special_chars])
link_count = np.concatenate([spam_link_count, ham_link_count])
label = np.concatenate([np.ones(n_samples // 2), np.zeros(n_samples // 2)]) # 1

data = pd.DataFrame({
    'word_frequency': word_frequency,
    'message_length': message_length,
    'special_char_count': special_char_count,
    'link_count': link_count,
    'label': label.astype(int)
})
```

```
In [3]: # Display the first few rows
print(data.shape)
data.head()
```

(500, 5)

```
Out[3]:
```

	word_frequency	message_length	special_char_count	link_count	label
0	77.450712	199.129284	19.630888	3.153534	1
1	67.926035	373.428956	24.547083	2.296631	1
2	79.715328	469.772496	8.007162	3.048253	1
3	92.845448	382.597221	17.814846	7.107284	1
4	66.487699	178.450403	11.746787	3.101202	1

```
In [4]: # Checking for any null values in dataset
data.isnull().sum()
```

```
Out[4]: word_frequency      0
message_length      0
special_char_count    0
link_count           0
label               0
dtype: int64
```

```
In [5]: # Assigning dependent and independent variables
feature_cols = ['word_frequency', 'message_length', 'special_char_count', 'link_
x = data[feature_cols]
y = data.label
```

```
In [6]: # Splitting the dataset into Training and Testing Dataset
x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.3, random_state=5)

display(x_train.shape, y_train.shape, x_test.shape,
y_test.shape)
```

```
(350, 4)
(350,)
(150, 4)
(150,)
```

```
In [7]: # Preprocessing Data with StandardScaler
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)
```

```
In [8]: # Fitting the Model (SVM) using 'rbf' kernel
model = SVC(kernel='rbf', random_state=0)
model.fit(x_train, y_train)
svc_prediction = model.predict(x_test)
print('svc_prediction: ', svc_prediction)
```

```
svc_prediction: [1 0 1 1 0 0 0 0 1 1 1 0 0 1 0 0 1 1 1 0 0 1 0 0 1 1 0 1 1 0 0 1
0 1 0 1 0
0 0 0 0 1 1 0 1 1 0 0 1 1 0 0 1 1 0 1 1 1 0 1 0 0 0 0 0 1 0 0 0 1 0 0 0 1
1 0 1 0 0 0 1 1 0 1 1 1 1 0 1 1 1 1 0 0 0 1 1 0 1 1 1 0 1 0 1 1 1 0 0 0 1
0 1 1 0 0 1 0 0 0 1 0 1 1 1 1 0 0 0 0 0 1 0 0 0 1 1 0 0 1 1 1 0 1 1 0 1 1
0 1]
```

```
In [9]: # Evaluation Metrics for 'rbf' kernel
conf_mat = metrics.confusion_matrix(y_test, svc_prediction)
```

```

print('SVC [ kernel - rbf ]')
print('Confusion Matrix : \n', conf_mat)
Accuracy_score = metrics.accuracy_score(y_test, svc_prediction)
print('Accuracy Score : ', Accuracy_score)
print('Accuracy in Percentage : ', int(Accuracy_score * 100), '%')
print(classification_report(svc_prediction, y_test))

```

```

SVC [ kernel - rbf ]
Confusion Matrix :
[[75  0]
 [ 1 74]]
Accuracy Score : 0.9933333333333333
Accuracy in Percentage : 99 %

```

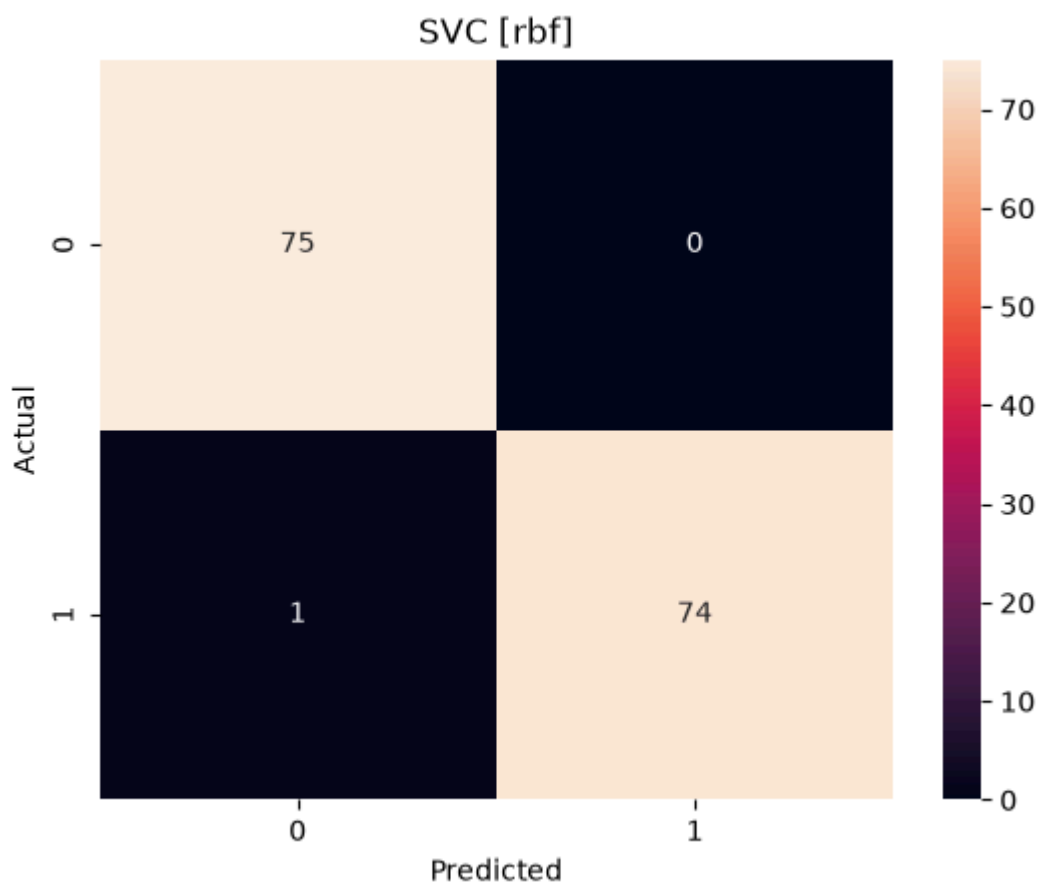
	precision	recall	f1-score	support
0	1.00	0.99	0.99	76
1	0.99	1.00	0.99	74
accuracy			0.99	150
macro avg	0.99	0.99	0.99	150
weighted avg	0.99	0.99	0.99	150

```

In [10]: # Confusion Matrix Heatmap for 'rbf' kernel
conf_mat = pd.crosstab(y_test, svc_prediction, rownames=['Actual'],
                        colnames=['Predicted'])
sn.heatmap(conf_mat, annot=True).set_title('SVC [rbf]')

```

Out[10]: Text(0.5, 1.0, 'SVC [rbf]')



```

In [11]: # Fitting the Model (SVM) using 'linear' kernel
model = SVC(kernel='linear', random_state=0)
model.fit(x_train, y_train)

```

```
svc_prediction = model.predict(x_test)
print('svc_prediction: ', svc_prediction)
```

```
svc_prediction: [1 0 1 1 0 0 0 0 1 1 1 0 0 1 0 0 1 1 1 0 0 1 0 0 1 1 0 1 1 0 0 1
0 1 0 1 0
0 0 0 0 1 1 0 1 1 0 0 1 1 0 0 1 1 0 1 1 1 0 1 0 0 0 0 0 1 0 0 0 1 0 0 0 1
1 0 1 0 0 0 1 1 0 1 1 1 1 0 1 1 1 1 0 0 0 1 1 0 1 1 1 0 1 0 1 1 1 0 0 0 1
0 1 1 0 0 1 0 0 0 1 0 1 1 1 1 0 0 0 0 0 1 0 0 0 1 1 0 0 1 1 1 0 1 1 0 1 1
0 1]
```

```
In [12]: # Evaluation Metrics for 'linear' kernel
conf_mat = metrics.confusion_matrix(y_test, svc_prediction)
print('SVC [ kernel - linear ]')
print('Confusion Matrix : \n', conf_mat)
Accuracy_score = metrics.accuracy_score(y_test, svc_prediction)
print('Accuracy Score : ', Accuracy_score)
print('Accuracy in Percentage : ', int(Accuracy_score * 100), '%')
print(classification_report(svc_prediction, y_test))
```

```
SVC [ kernel - linear ]
Confusion Matrix :
[[75  0]
 [ 1 74]]
Accuracy Score : 0.9933333333333333
Accuracy in Percentage : 99 %
```

	precision	recall	f1-score	support
0	1.00	0.99	0.99	76
1	0.99	1.00	0.99	74
accuracy			0.99	150
macro avg	0.99	0.99	0.99	150
weighted avg	0.99	0.99	0.99	150

```
In [13]: # Confusion Matrix Heatmap for 'linear' kernel
conf_mat = pd.crosstab(y_test, svc_prediction, rownames=['Actual'],
                        colnames=['Predicted'])
sn.heatmap(conf_mat, annot=True).set_title('SVC [linear]')
```

```
Out[13]: Text(0.5, 1.0, 'SVC [linear]')
```

